# Jogo de Assaltos Cooperativo (Roblox)

Documento de projeto para a fase de programação · 12 de outubro de 2026

## 1. A ideia

Jogo cooperativo de assaltos para 1 a 4 jogadores, inspirado no género PayDay, mas **original**: nome, gangue, máscaras e música próprios, sem usar nada da marca PayDay (Starbreeze).

**Ciclo de jogo:** entrar no lobby com a equipa > escolher o assalto > tentar em furtivo > se o alarme disparar, aguentar as vagas de polícia > levar os sacos até à carrinha > receber dinheiro e XP > comprar equipamento e máscaras no esconderijo > próximo assalto.

- **Assaltos curtos** (5 a 10 minutos), para dar vontade de fazer "só mais um".
- **Dois caminhos:** furtivo (guardas, câmaras, civis, cabos, cofres) ou barulhento (broca, vagas de polícia).
- **Sacos de dinheiro:** carregar um saco deixa o jogador mais lento; entregar na carrinha soma ao total.
- **Progressão:** dinheiro, níveis, armas, máscaras personalizáveis, desafios diários e semanais.
- **Social:** bónus por jogar com amigos; o cooperativo é o trunfo do jogo.

## 2. Regras de ouro técnicas

- **O servidor manda.** Dinheiro, alarme, danos, sacos e compras são decididos no servidor. O cliente só mostra coisas e pede ações.
- **Nunca confiar no cliente.** Cada RemoteEvent valida: o jogador está vivo? Está perto do objeto? O assalto está no estado certo? Já fez isto há pouco (cooldown)?
- **Um sistema = um ModuleScript.** Scripts pequenos e com uma só responsabilidade.
- **Configuração em tabelas** (valores dos sacos, preços, tempos), nunca números espalhados pelo código.
- **Pequeno e completo antes de grande:** um assalto a funcionar do início ao fim antes do segundo.

## 3. Estrutura do projeto no Roblox Studio

```
ReplicatedStorage/
  Shared/
    Config.lua            -- valores: sacos, preços, tempos, dificuldades
    HeistState.lua        -- nomes dos estados (Lobby, Furtivo, Alarme, Fuga, Fim)
  Remotes/                -- RemoteEvents e RemoteFunctions (ver secção 6)
ServerScriptService/
  Main.server.lua         -- arranca os serviços pela ordem certa
  Services/
    HeistService.lua      -- máquina de estados do assalto
    AlarmService.lua      -- deteção e disparo do alarme
    LootService.lua       -- sacos: apanhar, largar, entregar
    GuardService.lua      -- patrulha e visão dos guardas
    PoliceService.lua     -- vagas de polícia
    DrillService.lua      -- broca e encravamentos
    WireMinigame.lua      -- minijogo do cabo (validação no servidor)
    DataService.lua       -- DataStore: guardar e carregar perfis
    ShopService.lua       -- loja com dinheiro do jogo
    MonetizationService.lua -- Game Passes e Developer Products
StarterPlayer/StarterPlayerScripts/
  Client.client.lua       -- arranca os controladores do cliente
  Controllers/
    HUDController.lua     -- dinheiro, estado, temporizador
    InteractController.lua-- prompts e animações locais
    MinigameUI.lua        -- interface do cabo e do hacking
Workspace/
  Map/ (Joalharia)  Guards/  Loot/  Cameras/  EscapeZone/  Waypoints/
```

## 4. Estados do assalto (HeistService)

O assalto é uma máquina de estados controlada só pelo servidor. Cada mudança é enviada aos clientes pelo Remote **HeistStateChanged**.

| Estado | Entra quando | O que acontece | Sai para |
|---|---|---|---|
| Lobby | Servidor arranca | Jogadores juntam-se; o anfitrião escolhe dificuldade | Furtivo (todos prontos) |
| Furtivo | Assalto começa | Guardas patrulham, câmaras ativas, minijogos disponíveis | Alarme ou Fuga |
| Alarme | Guarda vê, câmara vê, cabo errado, civil foge | Vagas de polícia, broca ativa, portas trancadas | Fuga |
| Fuga | Saco mínimo entregue ou objetivo cumprido | Carrinha chega; temporizador para sair | Fim |
| Fim | Todos na carrinha, todos presos ou tempo esgotado | Ecrã de resultados; pagar dinheiro e XP | Lobby |

## 5. Sistemas e responsabilidades

| Módulo | Responsabilidade | Pontos-chave |
|---|---|---|
| AlarmService | Única porta de entrada para disparar o alarme | Função Trigger(motivo); ignora se já está em Alarme |
| GuardService | Patrulha e deteção | PathfindingService entre waypoints; visão = ângulo (ex.: 90º) + distância + raycast sem obstáculos; barra de suspeita antes de dar o alarme |
| LootService | Sacos de dinheiro | Um saco por jogador (2 com Game Pass); WalkSpeed reduzido; validar distância ao entregar |
| WireMinigame | Cortar o fio certo | Servidor escolhe o fio certo e guarda-o; cliente só envia o fio escolhido; erro = alarme, ou usar kit de emergência |
| DrillService | Broca no cofre | Temporizador no servidor; encrava aleatoriamente; jogador perto repara com E |
| PoliceService | Vagas depois do alarme | Vagas cada vez maiores; pontos de aparecimento fora da vista |
| DataService | Perfis guardados | Carregar ao entrar, guardar ao sair e de 2 em 2 min; pcall e novas tentativas; versão do perfil |
| MonetizationService | Compras | MarketplaceService; ProcessReceipt devolve PurchaseGranted só depois de guardar |

## 6. Comunicação cliente-servidor (Remotes)

| Remote | Direção | O servidor valida |
|---|---|---|
| HeistStateChanged | Servidor > todos | (só envia) |
| RequestInteract(objeto) | Cliente > servidor | Distância, jogador vivo, estado do assalto, cooldown |
| PickUpBag / DropBag | Cliente > servidor | Saco existe, livre, perto; limite de sacos |
| WireChoice(cor) | Cliente > servidor | Minijogo aberto para este jogador; uma tentativa |
| UseEmergencyKit | Cliente > servidor | Tem kit no perfil; houve erro há poucos segundos |
| UpdateHUD | Servidor > cliente | (só envia: dinheiro, temporizador, sacos) |

## 7. Dados guardados por jogador (DataStore)

```
Perfil = {
  versao = 1,
  dinheiro = 0,          -- dinheiro do jogo
  xp = 0, nivel = 1,
  kitsEmergencia = 0,    -- comprados ou ganhos
  armas = { "Pistola" },
  mascaras = { "Basica" },
  mascaraEquipada = "Basica",
  desafios = { diario = {}, semanal = {} },
  ultimaRecompensaDiaria = 0,  -- os.time()
  estatisticas = { assaltos = 0, furtivos = 0 },
}
```

## 8. Especificação do primeiro assalto: Joalharia

| Elemento | Valor inicial (ajustar nos testes) |
|---|---|
| Jogadores | 1 a 4 |
| Duração alvo | 5 a 10 minutos |
| Guardas | 3 a 4, com rotas fixas |
| Câmaras | 1 a 2 (desligam-se no painel de segurança com o minijogo do cabo) |
| Civis | 2 a 3 (se fugirem, dão o alarme) |
| Saque | Montras (sacos pequenos) + cofre (sacos grandes, precisa de broca no modo barulhento) |
| Fuga | Carrinha nas traseiras; mínimo de 2 sacos para chamar a fuga |
| Derrota | Jogador derrubado pode ser reanimado; todos presos = falhanço |

## 9. Monetização (entra na Fase 5)

Princípio: as compras são atalhos, nunca obrigação. Quem não paga consegue terminar todos os assaltos.

| Item | Tipo | Preço indicativo | Ganha-se a jogar? |
|---|---|---|---|
| Kit de emergência (segunda tentativa no cabo/hacking) | Developer Product | 25 a 50 Robux | Sim |
| Advogado (perdes menos se fores preso) | Developer Product | 25 a 50 Robux | Sim |
| Carregar 2 sacos / mais rápido com sacos | Game Pass | 149 a 299 Robux | Não |
| Dinheiro x2 | Game Pass | 299 a 499 Robux | Não |
| VIP (nome colorido, máscaras e esconderijo) | Game Pass | 199 a 399 Robux | Não |
| Máscaras, fatos, efeitos | Cosmético | Variável | Alguns |
| Servidor privado | Servidor privado | 50 a 100 Robux/mês | Não |

Cuidados: caixas aleatórias pagas têm de mostrar probabilidades; muitos jogadores são crianças, por isso nada de temporizadores falsos nem pop-ups de compra a cada falhanço.

## 10. Fases do projeto

### Fase 0 · Preparação (2 a 3 semanas)

☐ Instalar o Studio, explorar Explorer, Properties e Output ☐ Luau: variáveis, if, ciclos, funções, tabelas, eventos ☐ Script vs LocalScript; primeiro script (porta com E) ☐ Escolher nome do jogo e da gangue

**Concluída quando:** Crias e ligas um script a um objeto sem seguir um tutorial passo a passo.

### Fase 1 · Mecânicas base (4 a 6 semanas)

☐ Saco de dinheiro ☐ Zona de fuga ☐ Alarme ☐ Minijogo do cabo ☐ Broca ☐ Guarda que patrulha ☐ Guarda que vê ☐ Vagas de polícia ☐ HUD

**Concluída quando:** Todas as peças funcionam sozinhas, com a lógica no servidor.

### Fase 2 · Primeiro assalto completo (3 a 4 semanas)

☐ Mapa da joalharia ☐ Lobby ☐ Caminhos furtivo e barulhento ☐ Ecrã final ☐ Derrota e reanimar

**Concluída quando:** 4 amigos jogam do início ao fim sem bugs graves, nos dois modos.

### Fase 3 · Progressão (3 a 4 semanas)

☐ DataStore ☐ Níveis e XP ☐ Loja e máscaras ☐ Recompensa diária e desafios ☐ Segundo assalto ☐ Dificuldades

**Concluída quando:** O progresso fica guardado e há sempre algo novo para desbloquear.

### Fase 4 · Testes e polimento (2 a 3 semanas)

☐ Testes com amigos ☐ Tutorial ☐ Sons e iluminação ☐ Telemóvel e tablet ☐ Testes de segurança ☐ Ícone e miniaturas

**Concluída quando:** Alguém novo percebe o jogo sem ajuda e quer jogar outra vez.

### Fase 5 · Monetização (2 semanas)

☐ Criar produtos e passes ☐ MarketplaceService e ProcessReceipt ☐ Ligar o kit ao minijogo do cabo

**Concluída quando:** Compras funcionam em teste e nenhuma é obrigatória.

### Fase 6 · Lançamento (contínuo)

☐ Publicar ☐ Grupo Roblox e Discord ☐ Estatísticas de retenção (dia 1 e dia 7) ☐ Atualização a cada 3 a 4 semanas

**Concluída quando:** Os jogadores voltam no dia seguinte sem ser preciso lembrá-los.

## 11. Primeiro script: porta com a tecla E

Part chamada "Porta" com Anchored ativo, um ProximityPrompt e um Script dentro:

```
local porta = script.Parent
local prompt = porta.ProximityPrompt
local aberta = false

prompt.ActionText = "Abrir"

prompt.Triggered:Connect(function(jogador)
    aberta = not aberta
    porta.Transparency = aberta and 0.7 or 0
    porta.CanCollide = not aberta
    prompt.ActionText = aberta and "Fechar" or "Abrir"
end)
```

## 12. Referências

- Documentação oficial: create.roblox.com/docs (Luau, PathfindingService, DataStoreService, MarketplaceService)
- Exemplos do género para estudar: Entry Point, Notoriety
- Plano vivo do projeto (com caixas para marcar): o documento "Plano do jogo de assaltos (Roblox)" no Claude